

Step 1

1. Ledoit–Wolf Covariance Shrinkage

Ledoit–Wolf shrinkage combines the sample covariance matrix with a simpler target, usually a scaled identity matrix. Shrinking the covariance estimates toward this target reduces the effect of sampling noise and can produce more stable portfolio weights.

Evaluation:

The LLM's explanation is correct. One important nuance is that shrinkage affects the entire covariance matrix, including its eigenvalues, rather than only the off-diagonal covariance estimates. The standard Ledoit–Wolf method also estimates the shrinkage intensity from the data, so it generally should not be chosen manually.

2. Minimum-Variance Portfolio with Ledoit–Wolf Covariance

This approach constructs the minimum-variance portfolio using a Ledoit–Wolf covariance matrix instead of the ordinary sample covariance matrix. It does not require expected-return estimates, so it avoids one of the largest sources of error in the classical tangency portfolio.

Evaluation: The LLM's explanation is correct, but the method does not remove estimation error completely since covariances should still be estimated and may vary over time. It can also produce large or negative weights if short selling is allowed and no position or turnover constraints are imposed.

3. Black–Litterman

Black–Litterman begins with expected returns implied by the market portfolio and then combines them with the investor's views using a Bayesian framework. Compared with using historical average returns directly, it usually produces more stable expected-return estimates and less extreme portfolio weights.

Evaluation: The LLM's explanation is correct, but it leaves out a few inputs and assumptions. Yet, it did not consider the fact that Black–Litterman is not parameter-free. It requires market-cap or benchmark weights, a risk-aversion assumption, investor views, and confidence levels for those views, and the results may depend on these choices. Because the provided dataset does not contain market-cap weights for the ten industries, I would treat Black–Litterman as a research-only option unless an additional reliable data source is used.

Step 2

Choice of Robust Method

I chose the minimum-variance portfolio with Ledoit–Wolf covariance because it addresses both sources of instability highlighted in Question 1: it improves covariance estimation through shrinkage and avoids noisy expected-return estimates entirely. Compared with using Ledoit–Wolf covariance in a mean-return-based portfolio, the MVP removes the additional instability coming from estimated expected returns. I did not choose Black–Litterman because the dataset does not provide the market-cap weights or investor views required for implementation without additional assumptions.

Prompt Used

"Using the monthly returns for the ten industries, write concise Python code that estimates a Ledoit–Wolf covariance matrix and constructs a fully invested minimum-variance portfolio. Allow short selling so that the constraints are consistent with the unconstrained classical tangency portfolio. Estimate the shrinkage intensity automatically, report the robust portfolio weights, and compare their dispersion with the classical tangency weights. Explain the parameter and constraint choices and include checks that the optimization was completed correctly."

Parameter Choices

I used the standard LedoitWolf estimator, therefore the shrinkage intensity was estimated automatically rather than chosen manually. The portfolio was constrained to be fully invested, with weights summing to one, while short selling was allowed to remain consistent with the unconstrained tangency portfolio in Question 1(a). I used SLSQP for the minimum-variance optimization with an equal-weight starting portfolio, and converted returns from percentages to decimals before estimating the covariance matrices.

```
In [3]: import numpy as np
import pandas as pd
from scipy.optimize import minimize
from sklearn.covariance import LedoitWolf

FILE = "Problem_Set2_2026-1.xlsx"
industries = [
    "NoDur", "Durb1", "Manuf", "Enrgy", "HiTec",
    "Telcm", "Shops", "Hlth", "Utils", "Other"
]

df = pd.read_excel(FILE, sheet_name="Industry_returns", skiprows=20)
df.columns = df.columns.astype(str).str.strip()
df = df.rename(columns={df.columns[0]: "Date"})

data = df[industries + ["Risk-free rate"]]
data = data.apply(pd.to_numeric, errors="coerce")
data = data.replace([-99.99, -999], np.nan).dropna() / 100

R = data[industries]
```

```

rf = data["Risk-free rate"].mean()
n = len(industries)

lw = LedoitWolf().fit(R)
cov_lw = lw.covariance_

result = minimize(
    lambda w: w @ cov_lw @ w,
    np.ones(n) / n,
    method="SLSQP",
    constraints={"type": "eq", "fun": lambda w: w.sum() - 1},
    options={"ftol": 1e-14, "maxiter": 2000}
)

if not result.success:
    raise RuntimeError(result.message)

w_lw = result.x

mu = R.mean().to_numpy()
cov_sample = R.cov().to_numpy()

w_tan = np.linalg.solve(cov_sample, mu - rf)
w_tan /= w_tan.sum()

# Validation checks
w_check = np.linalg.solve(cov_lw, np.ones(n))
w_check /= w_check.sum()

assert np.isclose(w_lw.sum(), 1)
assert np.linalg.eigvalsh(cov_lw).min() > 0
assert np.allclose(w_lw, w_check, atol=1e-6)

weights = pd.DataFrame({
    "Classical tangency (%)": 100 * w_tan,
    "LW minimum variance (%)": 100 * w_lw,
    "Difference (pp)": 100 * (w_lw - w_tan)
}, index=industries)

dispersion = pd.DataFrame({
    "Classical tangency": [
        100 * w_tan.min(),
        100 * w_tan.max(),
        100 * w_tan.std(ddof=1),
        np.abs(w_tan).sum()
    ],
    "LW minimum variance": [
        100 * w_lw.min(),
        100 * w_lw.max(),
        100 * w_lw.std(ddof=1),
        np.abs(w_lw).sum()
    ]
}, index=[
    "Minimum weight (%)",
    "Maximum weight (%)",

```

```

    "Weight SD (percentage points)",
    "Gross exposure"
])

print(f"Observations used: {len(R)}")
print(f"Ledoit-Wolf shrinkage intensity: {lw.shrinkage_:.4f}")
print("\nPortfolio weights:")
print(weights.round(2))
print("\nWeight dispersion:")
print(dispersion.round(2))

```

Observations used: 1200

Ledoit-Wolf shrinkage intensity: 0.0139

Portfolio weights:

	Classical tangency (%)	LW minimum variance (%)	Difference (pp)
NoDur	66.45	64.65	-1.81
Durbl	13.40	-2.41	-15.81
Manuf	-22.42	-16.46	5.96
Enrgy	28.08	14.99	-13.10
HiTec	18.65	-7.26	-25.90
Telcm	15.25	48.38	33.14
Shops	7.25	1.18	-6.07
Hlth	35.30	13.08	-22.22
Utils	7.21	13.91	6.71
Other	-69.16	-30.07	39.10

Weight dispersion:

	Classical tangency	LW minimum variance
Minimum weight (%)	-69.16	-30.07
Maximum weight (%)	66.45	64.65
Weight SD (percentage points)	35.90	28.57
Gross exposure	2.83	2.12

Corrections and Validation

No substantive correction to the portfolio methodology was required. I used a tight optimization tolerance to ensure numerical precision and validated the result in three ways: the portfolio weights sum to one, the Ledoit-Wolf covariance matrix has strictly positive eigenvalues, and the optimized weights match the closed-form minimum-variance solution. These checks confirm that the optimization was implemented correctly.

Results

The Ledoit-Wolf estimator produced a shrinkage intensity of 0.0139, indicating that Ledoit-Wolf places most of the weight on the sample covariance matrix. With 1,200 observations for only 10 industries, this is reasonable because the covariance matrix is estimated from a relatively large sample, although it does not imply that the sample covariance matrix is free of estimation error.

The Ledoit–Wolf MVP has less dispersed weights than the classical tangency portfolio: the cross-sectional SD of the ten weights falls from 35.90 to 28.57 percentage points, gross exposure falls from 2.83 to 2.12, and the most negative position changes from –69.16% to –30.07%. This indicates a less extreme portfolio, although substantial long and short positions remain because short selling is unrestricted. Importantly, the difference should not be attributed entirely to covariance shrinkage: the tangency portfolio depends on both estimated expected returns and covariances, whereas the LW-MVP minimizes variance without using expected-return estimates. Therefore, this comparison measures the stability of the overall robust construction rather than the isolated effect of Ledoit–Wolf shrinkage.

Step 3 - Bootstrap Comparison

The following code uses the same empirical row-bootstrap engine as Question 1(e). Both portfolios are re-estimated from each bootstrap sample and then evaluated using the original industry-return moments.

```
In [ ]: from pathlib import Path
import matplotlib.pyplot as plt

import data_load as dl
import simulate as sim

N_SIMS = 1000
SEED = 20260918

d = dl.load()
names = d["names"]
R = d["R"]
mu = d["mu"]
Sigma = d["Sigma"]
rf = d["rf"]

def rule_lw_mvp(R_sim, rf):
    cov_lw = LedoitWolf().fit(R_sim).covariance_
    return dl.mvp_weights(cov_lw)

rules = {
    "Classical Tangency": sim.rule_tangency,
    "Ledoit–Wolf MVP": rule_lw_mvp,
}

results = sim.run(
    sim.make_bootstrap_sampler(R),
    rules,
    mu,
    Sigma,
    rf,
```

```

    n_sims=N_SIMS,
    seed=SEED,
)

actual_weights = {
    "Classical Tangency": dl.tangency_weights(mu, Sigma, rf),
    "Ledoit-Wolf MVP": dl.mvp_weights(LedoitWolf().fit(R).covariance_),
}

dispersion = {
    name: sim.dispersion(results[name], actual_weights[name], mu, Sigma, rf)
    for name in rules
}

weight_sd = pd.DataFrame(
    {name: results[name]["w"].std(axis=0, ddof=1) for name in rules},
    index=names,
)

weight_sd.loc["Average"] = weight_sd.mean()

metrics = pd.DataFrame({
    name: [
        100 * dispersion[name]["Avg cross-sim SD of weights"],
        dispersion[name]["SD of sim. portfolio mean"],
        dispersion[name]["SD of sim. portfolio SD"],
    ]
    for name in rules
}, index=[
    "Average weight SD (percentage points)",
    "SD of realized mean (%/month)",
    "SD of realized volatility (%/month)",
])

output_figs = Path("figs")
output_tables = Path("tables")
output_figs.mkdir(exist_ok=True)
output_tables.mkdir(exist_ok=True)

metrics.to_csv(output_tables / "q3_bootstrap_metrics.csv")
weight_sd.to_csv(output_tables / "q3_weight_sd.csv")
np.savez(
    output_tables / "q3_bootstrap_results.npz",
    w_tan=results["Classical Tangency"]["w"],
    w_lw=results["Ledoit-Wolf MVP"]["w"],
    mean_tan=results["Classical Tangency"]["mean"],
    sd_tan=results["Classical Tangency"]["sd"],
    mean_lw=results["Ledoit-Wolf MVP"]["mean"],
    sd_lw=results["Ledoit-Wolf MVP"]["sd"],
)

for name in rules:
    assert results[name]["w"].shape == (N_SIMS, len(names))
    assert np.allclose(results[name]["w"].sum(axis=1), 1, atol=1e-8)
    assert np.isfinite(results[name]["w"]).all()

    first_w = results[name]["w"][0]

```

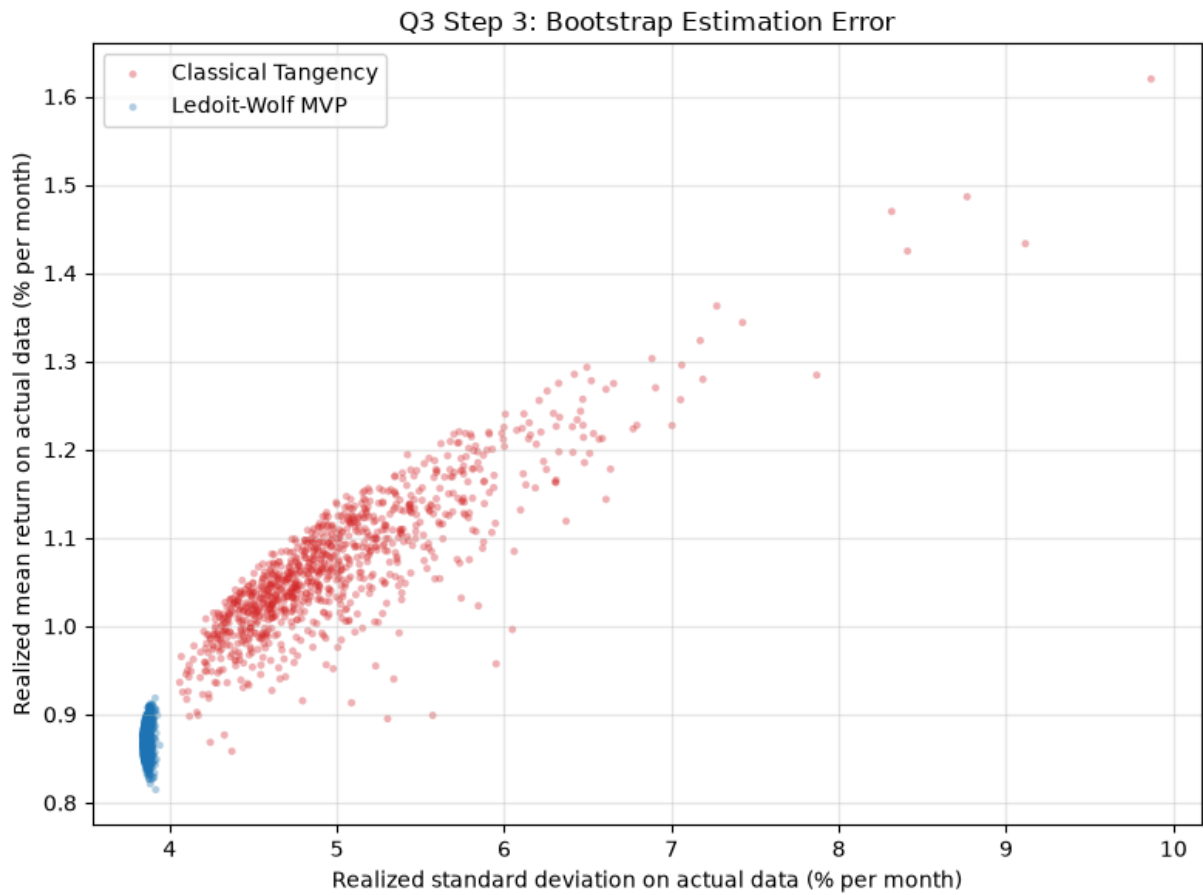
```

assert np.isclose(results[name]["mean"][0], first_w @ mu)
assert np.isclose(
    results[name]["sd"][0],
    np.sqrt(first_w @ Sigma @ first_w),
)

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(
    results["Classical Tangency"]["sd"],
    results["Classical Tangency"]["mean"],
    s=12,
    alpha=0.35,
    color="C3",
    edgecolors="none",
    label="Classical Tangency",
)
ax.scatter(
    results["Ledoit-Wolf MVP"]["sd"],
    results["Ledoit-Wolf MVP"]["mean"],
    s=12,
    alpha=0.35,
    color="C0",
    edgecolors="none",
    label="Ledoit-Wolf MVP",
)
ax.set_xlabel("Realized standard deviation on actual data (% per month)")
ax.set_ylabel("Realized mean return on actual data (% per month)")
ax.set_title("Q3 Step 3: Bootstrap Estimation Error")
ax.grid(alpha=0.3)
ax.legend()
fig.tight_layout()
fig.savefig(output_figs / "q3_bootstrap_comparison.png", dpi=200)
plt.show()

print(f"Simulations: {N_SIMS}; seed: {SEED}")
display(metrics.round(6))
display(weight_sd.round(6))

```



Simulations: 1000; seed: 20260918

	Classical Tangency	Ledoit-Wolf MVP
Average weight SD (percentage points)	27.027876	4.831692
SD of realized mean (%/month)	0.080225	0.016798
SD of realized volatility (%/month)	0.637692	0.015585

	Classical Tangency	Ledoit-Wolf MVP
NoDur	0.409253	0.063527
Durbl	0.168339	0.034145
Manuf	0.397474	0.070430
Enrgy	0.171395	0.030374
HiTec	0.228956	0.034845
Telcm	0.233406	0.045430
Shops	0.313933	0.048408
HLth	0.250919	0.043199
Utils	0.213361	0.040042
Other	0.315751	0.072769
Average	0.270279	0.048317

Step 3 Interpretation

The Ledoit-Wolf MVP is substantially more stable than the classical tangency portfolio across all three bootstrap measures. Average cross-simulation weight SD falls from 27.03 to 4.83 percentage points, while the SD of realized mean returns falls from 0.0802% to 0.0168% per month and the SD of realized volatility falls from 0.6377% to 0.0156% per month. This result is consistent with the fact that the tangency portfolio depends on noisy expected-return estimates as well as covariance estimates, whereas the LW-MVP avoids expected-return estimation and uses a shrinkage-based covariance estimate. Nevertheless, the improvement should not be attributed solely to Ledoit-Wolf shrinkage. This is due to the fact that changing from a tangency objective to an MVP objective also removes the especially noisy expected-return estimates.

Step 4 - Reflection

The Ledoit-Wolf MVP still relies on historical covariance estimates and leaves out expected returns. Thus greater stability may come at the cost of failing to exploit persistent return differences across industries. Shrinkage may also oversimplify the covariance structure if historical relationships contain useful information that the shrinkage target pulls away from. I would expect the method to provide more stable out-of-sample performance when expected returns are difficult to estimate, while it may underperform a well-specified tangency strategy when expected-return signals are persistent and reliable. In practice, I would use it as a robust baseline or core allocation, combined with position and turnover limits and regular out-of-sample monitoring rather than as a universally optimal portfolio.